Applied LLM Course

LangChain

From a first model call to tools, MCP and RAG

One example throughout: a bank’s customer assistant

LangChain is a framework for LLM apps

Three core packages, each built on the one below:

`langchain`
ready-made parts: init_chat_model, create_agent

`langgraph`
runs agents and workflows as graphs

`langchain-core`
the building blocks: messages, tools, Runnable

Provider packages

One per model provider. Install the one you use.

```
langchain-anthropic
langchain-openai
langchain-ollama
…
```

Runnable: every component has the same methods: invoke, stream, batch (and async versions). That is why parts fit together.

Open source (MIT license). Version 1.0 was released in October 2025.

Five steps, one running example

1	**Talking to a model**	Send messages, read the reply, keep a conversation, count the cost.
2	**Building with LangChain**	Connect steps into chains; turn a complaint into checked fields.
3	**Tool calling**	Let the model look up a customer’s transactions.
4	**MCP**	Connect tools through one standard protocol.
5	**RAG**	Answer from the bank’s own policy documents.

The example: a bank’s customer assistant that answers questions, sorts complaints, looks up transactions and answers from the bank’s policies.

1

Talking to a model

The smallest program, conversations, the reply object, streaming and cost.

A first LangChain program is three lines

LangChain

```python
from langchain.chat_models import init_chat_model
model = init_chat_model("anthropic:claude-sonnet-4-6")
print(model.invoke("Hello!").content)
```

OpenAI-compatible

```python
from openai import OpenAI
client = OpenAI()
r = client.chat.completions.create(model="gpt-5", messages=[{"role": "user", "content": "Hello!"}])
print(r.choices[0].message.content)
```

Install the provider package too: **pip install langchain langchain-anthropic**

Same length for simple tasks. LangChain pays off as tasks get more complex.

The messages list is the context




```
messages = [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "How do I dispute a card charge?"},
    AIMessage(content="Open the app, pick the charge, tap Dispute."),
    {"role": "user", "content": "How long does it take?"},
    AIMessage(content="Usually up to 10 working days."),
    {"role": "user", "content": "Thanks!"},
]
```

In your code, each reply is an AIMessage

`.content`	the text of the reply
`.usage_metadata`	token counts
`.response_metadata`	model, stop reason
`.tool_calls`	tool requests (none here)

On every call, the whole list is sent to the model. The model remembers nothing else.

A chat assistant is a loop

```python
from langchain.chat_models import init_chat_model
model = init_chat_model("anthropic:claude-sonnet-4-6")

messages = [{"role": "system", "content": "You are a helpful assistant."}]

for turn in range(10):  # up to 10 turns
    user_input = input("You: ")
    if user_input == "quit":
        break
    messages.append({"role": "user", "content": user_input})
    reply = model.invoke(messages)
    messages.append(reply)
    print("Assistant:", reply.content)
```

Each turn

1. Read input
2. Add the user message
3. Invoke with the whole history
4. Add the reply

The list is the assistant's only memory.

This is the list from the previous slide, built one turn at a time.

invoke() returns an AIMessage

LangChain's object for one model reply. Every chat API has an assistant message; this class is LangChain's version of it.

Field	Holds	Filled from the provider's
.content	the text of the reply	content
.tool_calls	tool requests (empty if none)	tool_use blocks
.usage_metadata	token counts for this call	usage
.response_metadata	model name, stop reason, id	model, stop_reason, id

```
reply = model.invoke(messages)
reply.content                          # a field: use a dot
reply.usage_metadata["input_tokens"]   # a key in a dict: use brackets
```

Also on every AIMessage: id, name, invalid_tool_calls, additional_kwargs (legacy), and the read-only .text and .content_blocks.

stream() delivers the reply piece by piece

invoke(): wait, then the whole reply

```
reply = model.invoke(question)
print(reply.content)
```

After a pause:

Open the app, pick the charge and tap Dispute.

stream(): pieces as they are generated

```
for chunk in model.stream(question):
    print(chunk.content, end="", flush=True)
```

Right away, one piece at a time:

Open the app, | pick the charge | and tap | Dispute.

A one-word change

stream instead of invoke. The user sees text immediately.

Each piece is a chunk

An AIMessageChunk. Chunks can be added together into one message.

Every Runnable has it

But only parts that support streaming deliver real pieces. A plain function returns one chunk.

Cost is counted in tokens, per call

Priced per token

Input and output tokens are priced separately; output usually costs more.

Reported per call

Each reply's usage_metadata holds the counts for that call only.

Chats grow

History is resent, so input tokens rise every turn.

```
total_in = total_out = 0
for each turn:                                      # pseudocode
    reply = model.invoke(messages)
    total_in  += reply.usage_metadata["input_tokens"]
    total_out += reply.usage_metadata["output_tokens"]

cost = total_in * PRICE_IN + total_out * PRICE_OUT   # prices per token, from the provider's price list
```

Shortcut: LangChain's get_usage_metadata_callback() collects the totals for every call made inside a with block.

2

Building with LangChain

Chains connect steps; structured output turns text into fields.

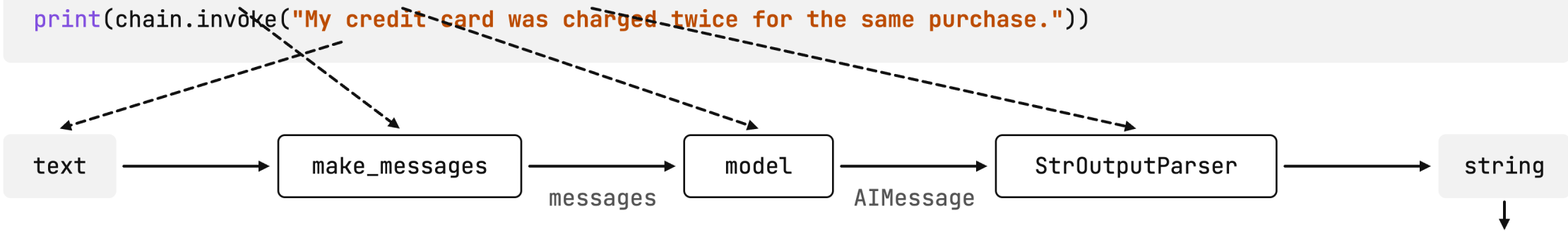
The | operator chains steps together

```python
from langchain.chat_models import init_chat_model
from langchain_core.output_parsers import StrOutputParser

def make_messages(complaint):
    return [
        {"role": "system", "content": "You are a bank's customer service assistant."},
        {"role": "user", "content": f"Write a short, polite reply to this complaint:\n{complaint}"},
    ]

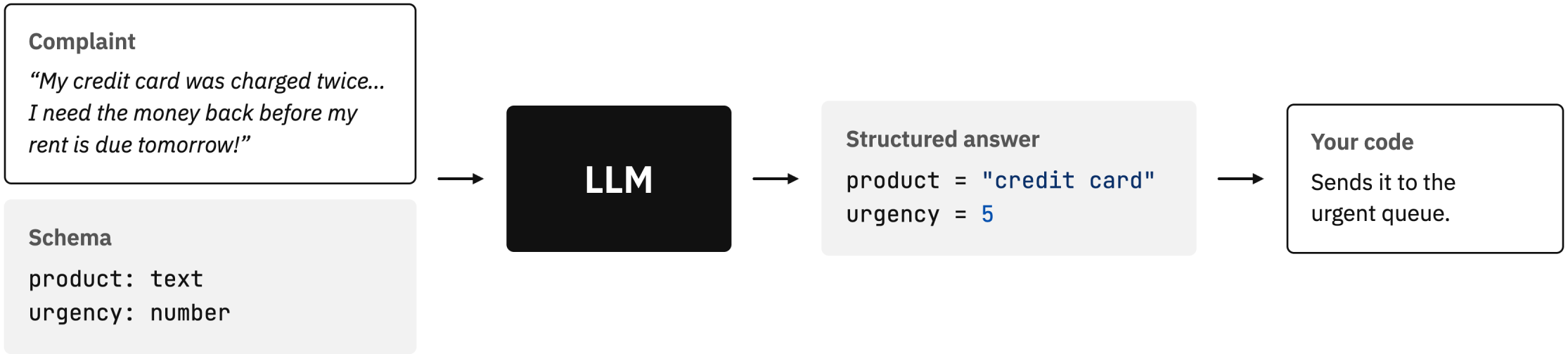
model = init_chat_model("anthropic:claude-sonnet-4-6")

chain = make_messages | model | StrOutputParser()
print(chain.invoke("My credit card was charged twice for the same purchase."))
```



“We’re sorry about the double charge on your card. We’ve started a refund for you.”

Structured output turns text into fields



Instead of free text: “This seems to be an urgent credit card issue.” A person can read it; code can’t act on it reliably.

1. Schema

Describe the shape of the answer: fields and types.

2. Model

The LLM fills in the fields from the text.

3. Parsing

The output becomes a typed, checked object.

A schema class and one method do the work

```python
from langchain.chat_models import init_chat_model
from pydantic import BaseModel

class Complaint(BaseModel):
    product: str
    urgency: int

model = init_chat_model("anthropic:claude-sonnet-4-6")
structured = model.with_structured_output(Complaint)

text = ("My credit card was charged twice for the same purchase. "
        "I need the money back before my rent is due tomorrow!")
result = model.with_structured_output(Complaint).invoke(text)
print(result)
```

One new LangChain call. Everything else is Pydantic or what you already know.

→ **Schema:** plain Pydantic, not LangChain

→ **New in LangChain:** with_structured_output wraps the model and returns a new Runnable

→ **Same invoke()** as before

Same thing in one line: structured is just a name for the wrapped model

```
product='credit card' urgency=5
```

Returns a **Complaint** object, not an AIMessage, so there's no .content

A Pydantic class becomes a JSON schema

In your code (Pydantic)

```python
from pydantic import BaseModel

class Complaint(BaseModel):
    product: str
    urgency: int
```


Sent to the model (JSON schema)

```json
{
  "title": "Complaint",
  "type": "object",
  "properties": {
    "product": {"title": "Product", "type": "string"},
    "urgency": {"title": "Urgency", "type": "integer"}
  },
  "required": ["product", "urgency"]
}
```

Pydantic is a library for describing the shape of data.

Each line in the class is one field: **name: type**.

Field names are your choice, but the model reads them.

That’s all the model gets: two names and two types. What “urgency” means is left to guess.

Descriptions and limits guide the model

Describe each field and limit its values

```python
from typing import Literal
from pydantic import BaseModel, Field

class Complaint(BaseModel):
    """A customer complaint received by the bank."""
    product: Literal["credit card", "loan", "account",
                     "mobile app", "other"] = Field(
        description="Banking product the complaint is about")
    urgency: int = Field(ge=1, le=5,
        description="How urgent: 1 = low, 5 = critical")
```

Four tools

`docstring`

describes the whole object

`description`

says what each field holds

`Literal[...]`

a fixed list of allowed values

`ge / le`

minimum and maximum

In the JSON schema these become description, enum, minimum and maximum: instructions, not guesswork.

The model sees the full schema

```
{
  "description": "A customer complaint received by the bank.",
  "properties": {
    "product": {
      "description": "Banking product the complaint is about",
      "enum": ["credit card", "loan", "account", "mobile app", "other"],
      "title": "Product", "type": "string"},
    "urgency": {
      "description": "How urgent: 1 = low, 5 = critical",
      "maximum": 5, "minimum": 1,
      "title": "Urgency", "type": "integer"}
  },
  "required": ["product", "urgency"],
  "title": "Complaint", "type": "object"
}
```

Same two fields as the simple schema. The parts in **bold orange** are new: description, enum, minimum and maximum.

The format is enforced in three places

1 · before the reply

The provider

The model is made to follow the schema: native structured output, or a forced tool call.

2 · after the reply

Pydantic

Checks the reply. urgency=7 or product="ATM" raises an error instead of slipping through.

3 · in your prompt

Your instructions

Rules that need judgement. The schema can’t say “fraud is always urgent”; the system prompt can.

```
result = structured.invoke([
    {"role": "system", "content": "Classify bank complaints. Fraud is always urgency 5."},
    {"role": "user", "content": complaint},
])
```

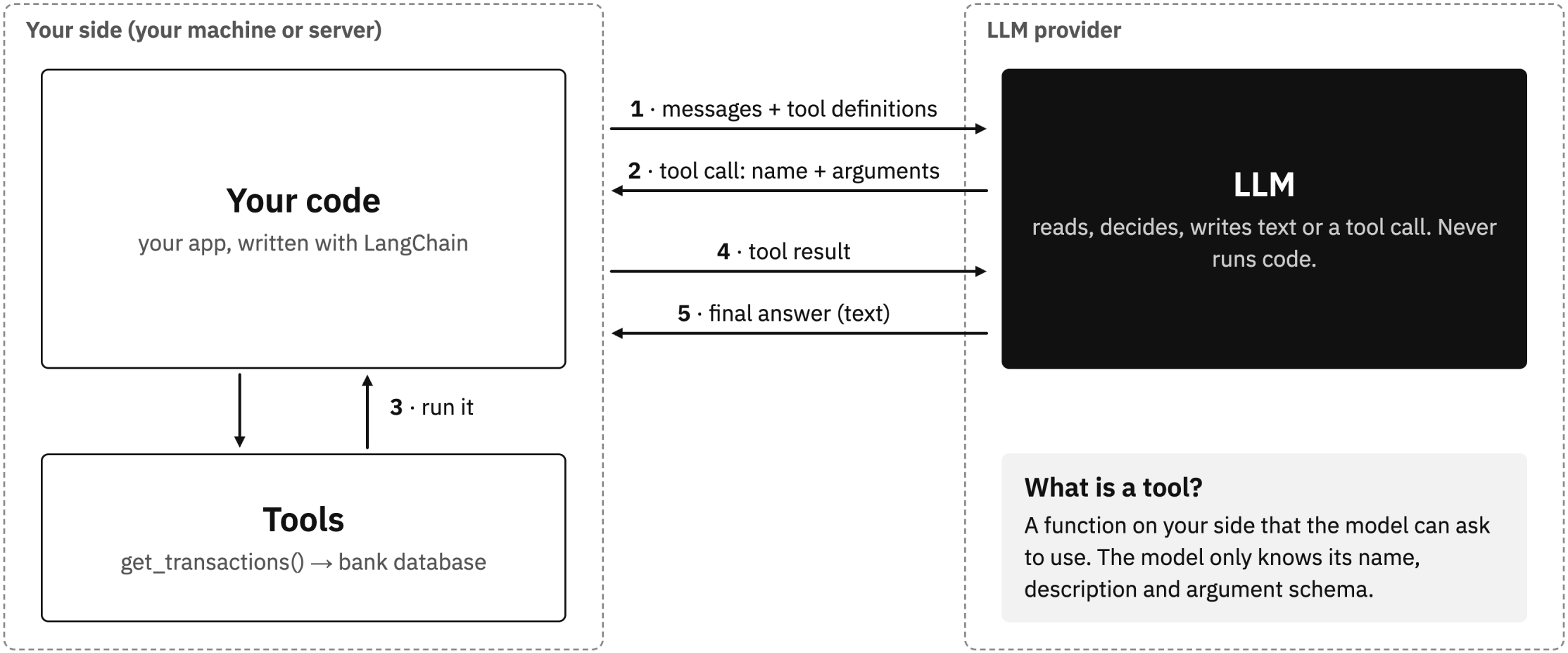
System prompt: fixed rules, written once. **User message:** the data, different on every call.

3

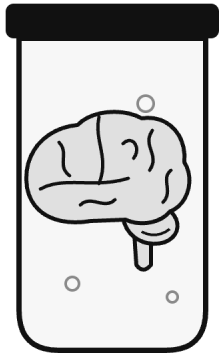
Tool calling

The model asks for a tool; your code runs it.

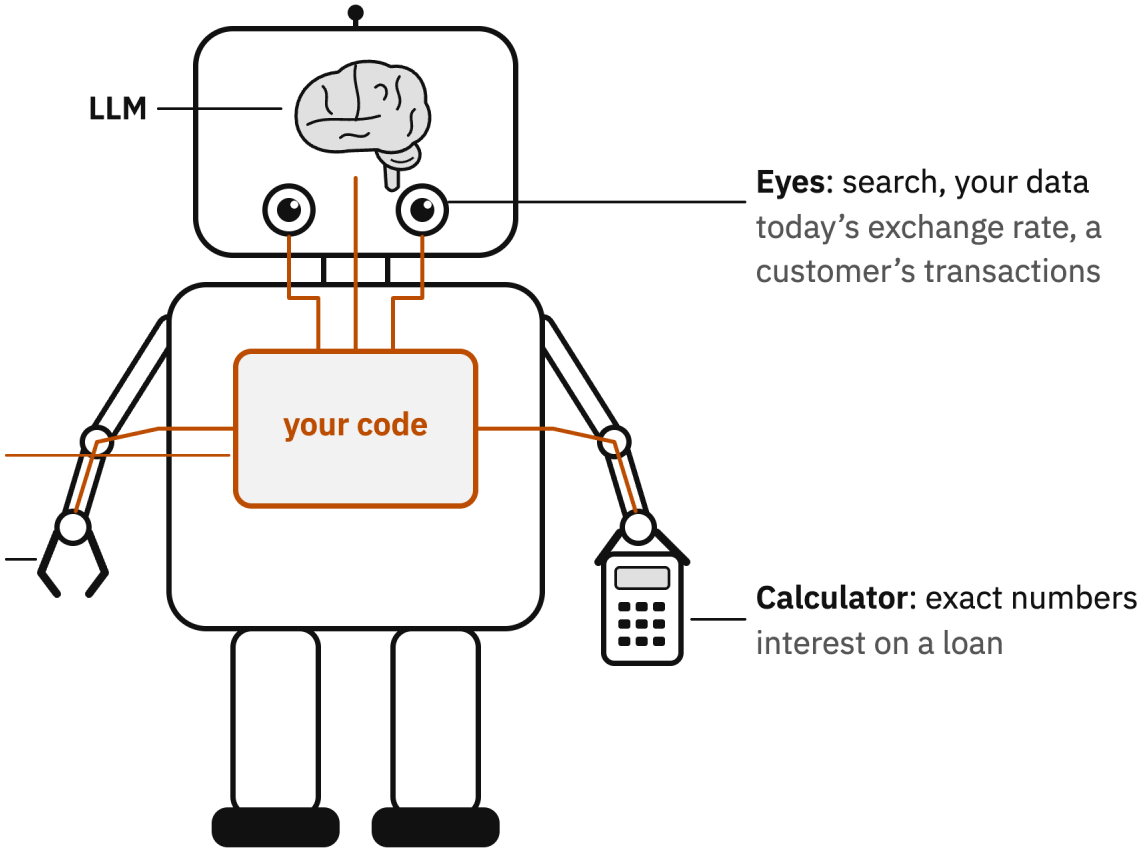
The LLM asks, your code runs the tool



Tools let the model see and act



LLM alone: a brain in a jar.
It can think and talk, but its knowledge stops at training time and it can't see or do anything.



Eyes: search, your data
today's exchange rate, a customer's transactions

Wires = your code
every request goes through it, and it can refuse

Hands: actions
block a card, open a ticket

Calculator: exact numbers
interest on a loan

A tool is a name, a description and a schema

In your code

```python
def get_transactions(account_id: str, days: int) → list:
    """Return the account's transactions from the last N days."""
    ... # your code: query the bank's database
```

The model never sees this code, only the definition below.

Sent to the model

```json
{
  "name": "get_transactions",
  "description": "Return the account's transactions from the last N days.",
  "input_schema": {"type": "object",
    "properties": {"account_id": {"type": "string"}, "days": {"type": "integer"}},
    "required": ["account_id", "days"]}
}
```

name: how the model refers to the tool

description: what it does; the model decides from this

input_schema: the arguments, as a JSON schema

Structured output used the same mechanism: the **Complaint** schema was sent to the model as a tool.

Tools are sent with every request

Sent on every call

One request

system
the assistant’s instructions

tools
get_transactions: name, description, argument schema

messages
the conversation so far

Kept on your side

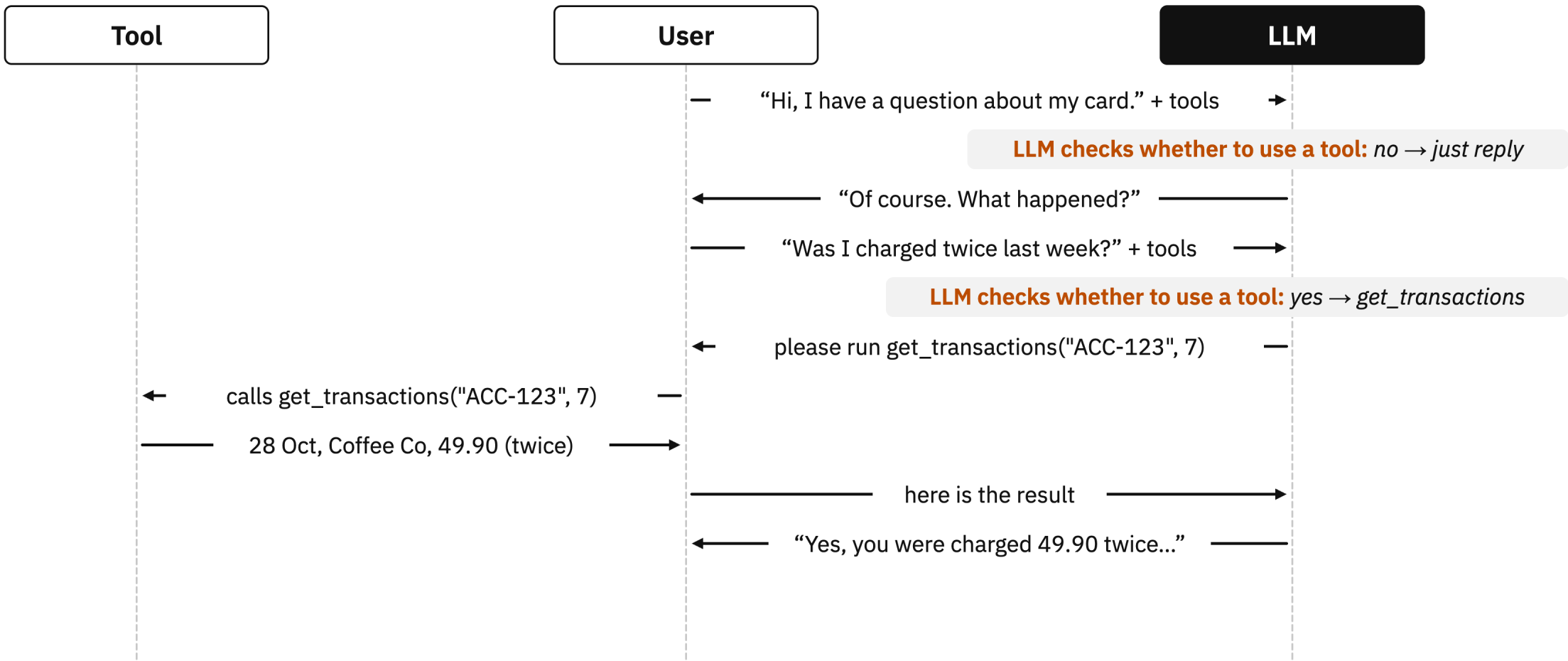
`"get_transactions"`
→ `get_transactions()`

The real function never leaves your code.

As it goes over the wire (JSON, Anthropic API)

```
{
  "model": "claude-sonnet-4-6", "max_tokens": 1024,
  "system": "You are a bank's customer service assistant.",
  "tools": [{
    "name": "get_transactions",
    "description":
      "Return the account's transactions from the last N days.",
    "input_schema": {"type": "object",
      "properties": {"account_id": {"type": "string"},
                     "days": {"type": "integer"}},
      "required": ["account_id", "days"]}
  }],
  "messages": [
    {"role": "user", "content": "Hi, I have a question about my card."},
    {"role": "assistant", "content": "Of course. What happened?"},
    {"role": "user", "content": "Was I charged twice last week?"}
  ]
}
```

A tool call takes two round trips



The model’s reply asks for a tool

① model asks for a tool → ② your code runs it → ③ result goes back → ④ model answers

The provider’s response (JSON)

```
{
  "stop_reason": "tool_use",
  "content": [
    {"type": "tool_use", "id": "toolu_01",
     "name": "get_transactions",
     "input": {"account_id": "ACC-123", "days": 7}}
  ]
}
```

→ **stop_reason**: the model stopped to wait for a tool

→ **id**: a ticket number; the result must quote it

→ **name + input**: which tool, with arguments the model filled in (“last week” → days: 7)

Nothing has run yet. This is only a request.

In LangChain this arrives as an AIMessage with .tool_calls = [{name, args, id}].

The result goes back as a tool_result

① model asks for a tool → ② your code runs it → ③ result goes back → ④ model answers

The next request's messages

```
"messages": [
  {"role": "user", "content": "Was I charged twice last week?"},
  {"role": "assistant", "content": [
    {"type": "tool_use", "id": "toolu_01", "name": "get_transactions",
     "input": {"account_id": "ACC-123", "days": 7}}]},
  {"role": "user", "content": [
    {"type": "tool_result", "tool_use_id": "toolu_01",
     "content": "28 Oct Coffee Co 49.90; 28 Oct Coffee Co 49.90"}]}
]
```

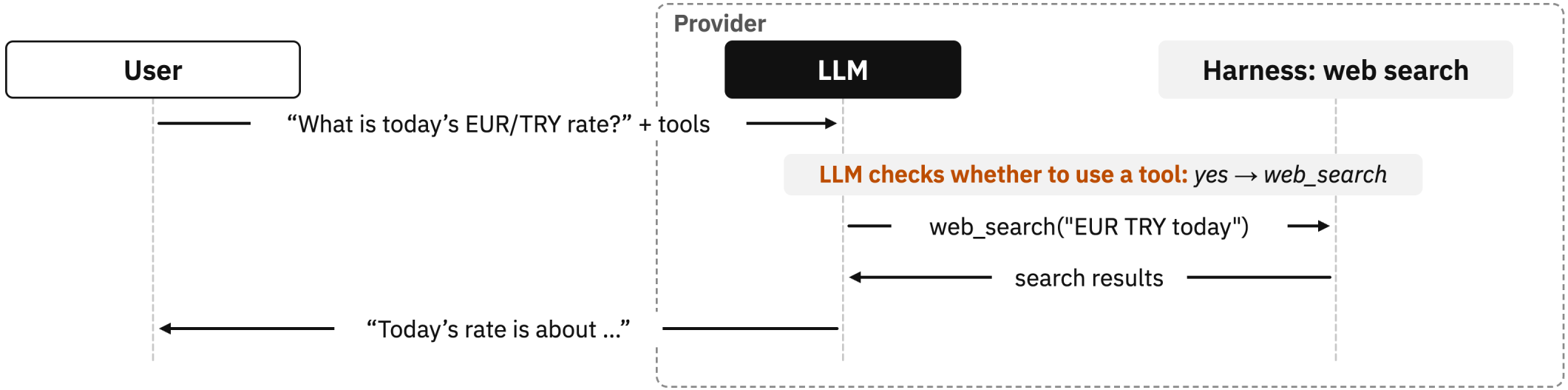
Same id on the call and the result: that's how the model matches them

Tool result: added by your code after running get_transactions

The model's final answer

"Yes, you were charged 49.90 twice at Coffee Co on 28 October. I'll flag it for a refund."

Some tools run on the provider's side



Your tools: you run them, with a round trip per call. **Server tools**: the provider runs them, all in one call.

Switch it on (in the request)

```
"tools": [{
  "type": "web_search_20260318",
  "name": "web_search", "max_uses": 3}]
```

What comes back (in the response)

```
"content": [
  {"type": "server_tool_use", "name": "web_search", ...},
  {"type": "web_search_tool_result", ...},
  {"type": "text", "text": "Today's rate is about …"}],
"stop_reason": "end_turn"
```

create_agent runs the tool loop for you

```python
from langchain.tools import tool
from langchain.agents import create_agent

@tool
def get_transactions(account_id: str, days: int) -> str:
    """Return the account's transactions from the last N days."""
    ... # query the bank's database

agent = create_agent("anthropic:claude-sonnet-4-6", tools=[get_transactions])
question = "Was I charged twice last week? My account is ACC-123."
result = agent.invoke({"messages": [{"role": "user", "content": question}]})
print(result["messages"][-1].content)
```

`@tool`

Name, docstring and type hints become the tool definition.

`create_agent`

Calls the LLM, runs the tools it asks for, sends the results back, until the answer.

`result["messages"]`

The whole exchange: question, tool call, tool result, answer.

4

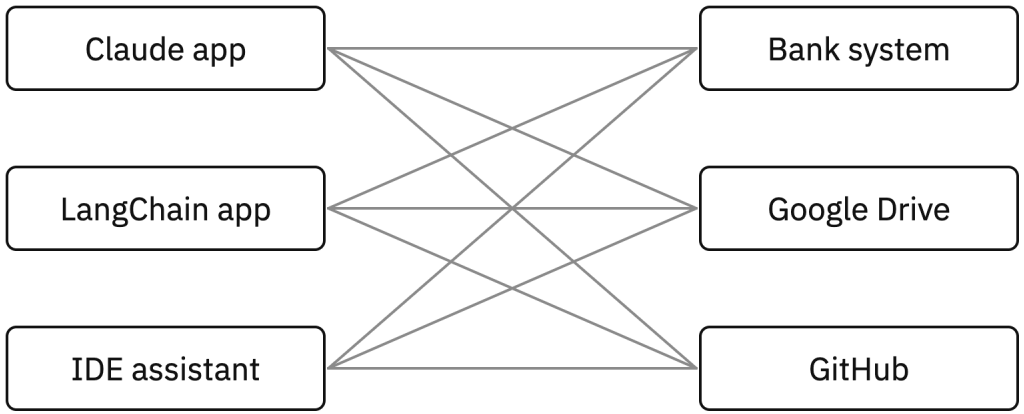
MCP

One standard way to connect apps and tools.

MCP connects any app to any tool

Without MCP

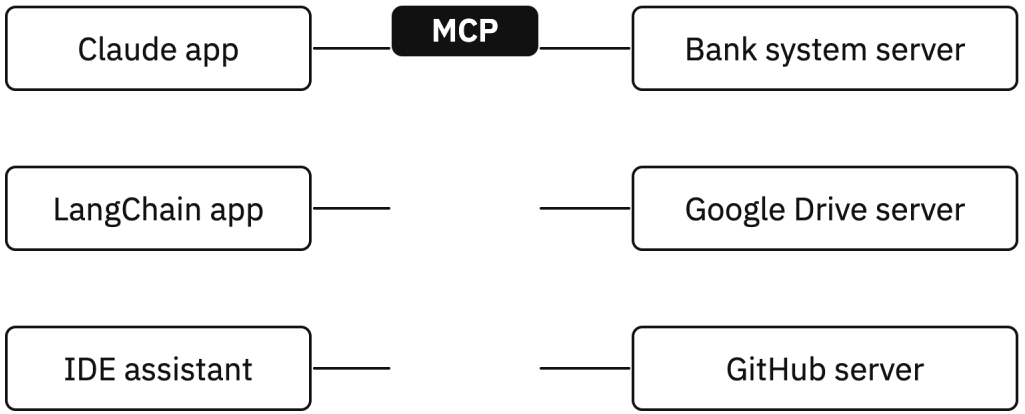
every app needs its own connection to every tool



3 apps × 3 tools = **9 custom integrations**

With MCP

each side implements MCP once

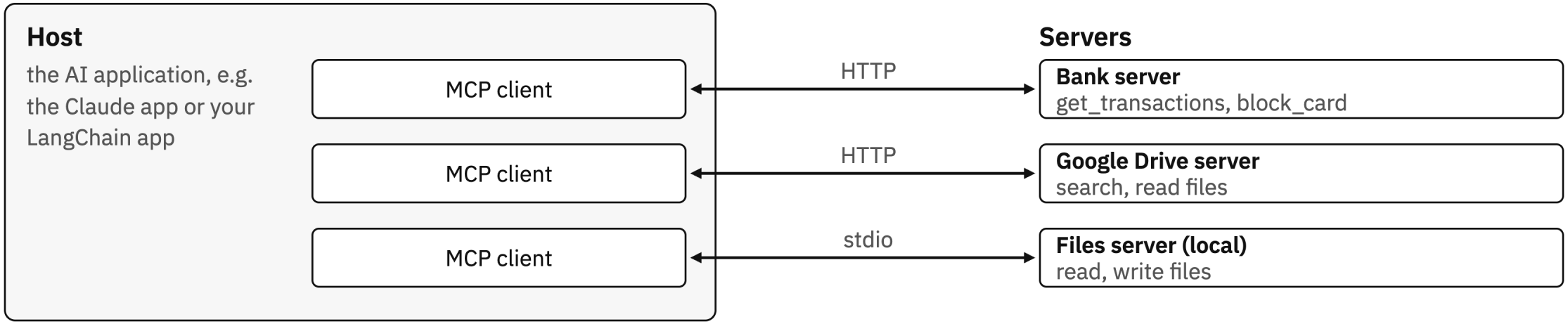


3 apps + 3 servers = **6 MCP connections**

The bank wraps get_transactions and block_card once, as an MCP server. **Every MCP-capable app can then use them.**

MCP has three roles: host, client, server

An **open protocol** for connecting AI applications to tools and data. Introduced by Anthropic in late 2024, now a Linux Foundation project.



Host
the app the user works in; it also talks to the LLM

Client
one per server, inside the host; speaks MCP

Server
offers tools, data and prompts to any host

A server offers tools, resources and prompts

Tools

controlled by the **model**

Functions the LLM can call.

Bank example:

```
get_transactions
```

Resources

controlled by the **app**

Data the app can read and add to the context, addressed by a URI.

Bank example:

```
account://ACC-123/statement
```

Prompts

controlled by the **user**

Ready-made prompt templates, e.g. offered as slash commands.

Bank example:

```
/dispute-charge
```

In practice, most servers offer mainly **tools**, and that is what the rest of this section is about.

On the wire, MCP is JSON-RPC

MCP messages are **JSON-RPC 2.0**. Two of them do most of the work. → host to server ← server to host

tools/list: “what tools do you have?”

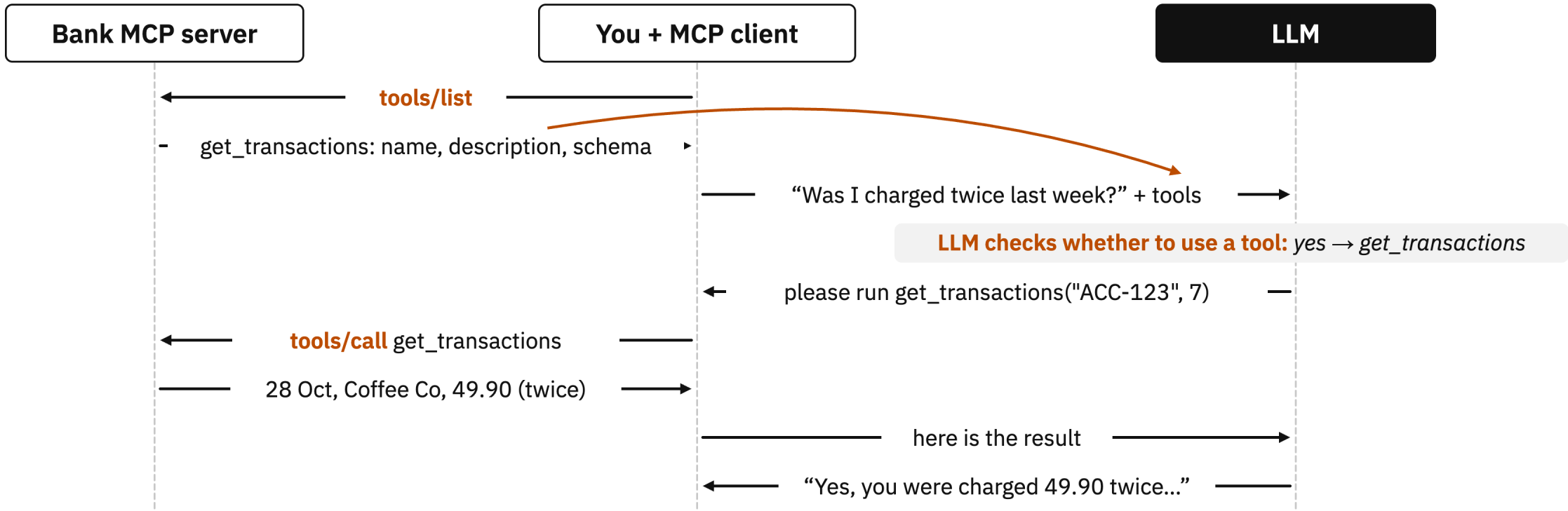
```
→ {"jsonrpc": "2.0", "id": 1, "method": "tools/list"}
← {"jsonrpc": "2.0", "id": 1, "result": {"tools": [
    {"name": "get_transactions", "description": "Return the account's transactions from the last N days.",
     "inputSchema": {"type": "object", "properties": {"account_id": {...}, "days": {...}}}}]}}
```

tools/call: “run this tool with these arguments”

```
→ {"jsonrpc": "2.0", "id": 2, "method": "tools/call",
    "params": {"name": "get_transactions", "arguments": {"account_id": "ACC-123", "days": 7}}}
← {"jsonrpc": "2.0", "id": 2, "result":
    {"content": [{"type": "text", "text": "28 Oct Coffee Co 49.90; 28 Oct Coffee Co 49.90"}]}}
```

Transports: stdio when the server runs as a local process, Streamable HTTP when it runs remotely.

MCP sits between your app and the tool



The LLM never speaks MCP. Your MCP client translates: tools/list → the LLM’s tools field, the LLM’s tool call → tools/call.

MCP servers are easy to use and to write

Use MCP servers from LangChain

```python
from langchain_mcp_adapters.client import MultiServerMCPClient
from langchain.agents import create_agent

client = MultiServerMCPClient({"bank": {"transport": "http", "url": "https://bank.example.com/mcp"}})
tools = await client.get_tools()          # tools/list → LangChain tools
agent = create_agent("anthropic:claude-sonnet-4-6", tools)
```

Write a server (Python, FastMCP)

```python
from mcp.server.fastmcp import FastMCP
mcp = FastMCP("bank")

@mcp.tool()
def get_transactions(account_id: str, days: int) -> str:
    """Return the account's transactions from the last N days."""
    ... # query the bank's database
mcp.run(transport="streamable-http")
```

In the Claude app

The **Connectors** directory is a catalogue of MCP servers. Connect one and its tools appear in your chats.

Only connect servers you trust. Their tool descriptions and results go straight into the model's context.

5

RAG

Answering from the bank’s own documents.

The model doesn’t know the bank’s own rules

The model alone

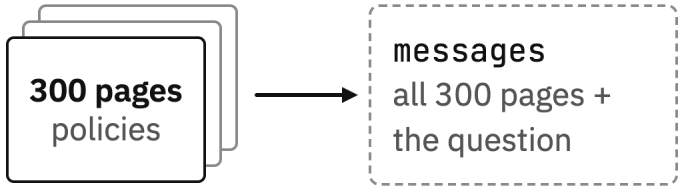
“What is the fee for a late credit card payment?”



“Late payment fees vary by bank and are often around…”

A generic guess. The model has never seen this bank’s fee schedule.

Paste every document in?



1 · Worse answers. The more text in the context, the less reliably the model uses it (“context rot”). Similar-looking passages distract it.

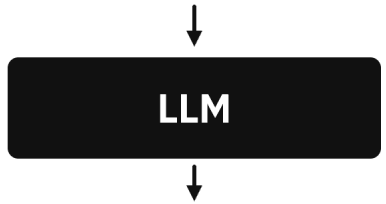
2 · More tokens on every call. Higher cost and longer waits.

3 · Large document sets don’t fit at all.

RAG: search first



only 3 passages go into the messages

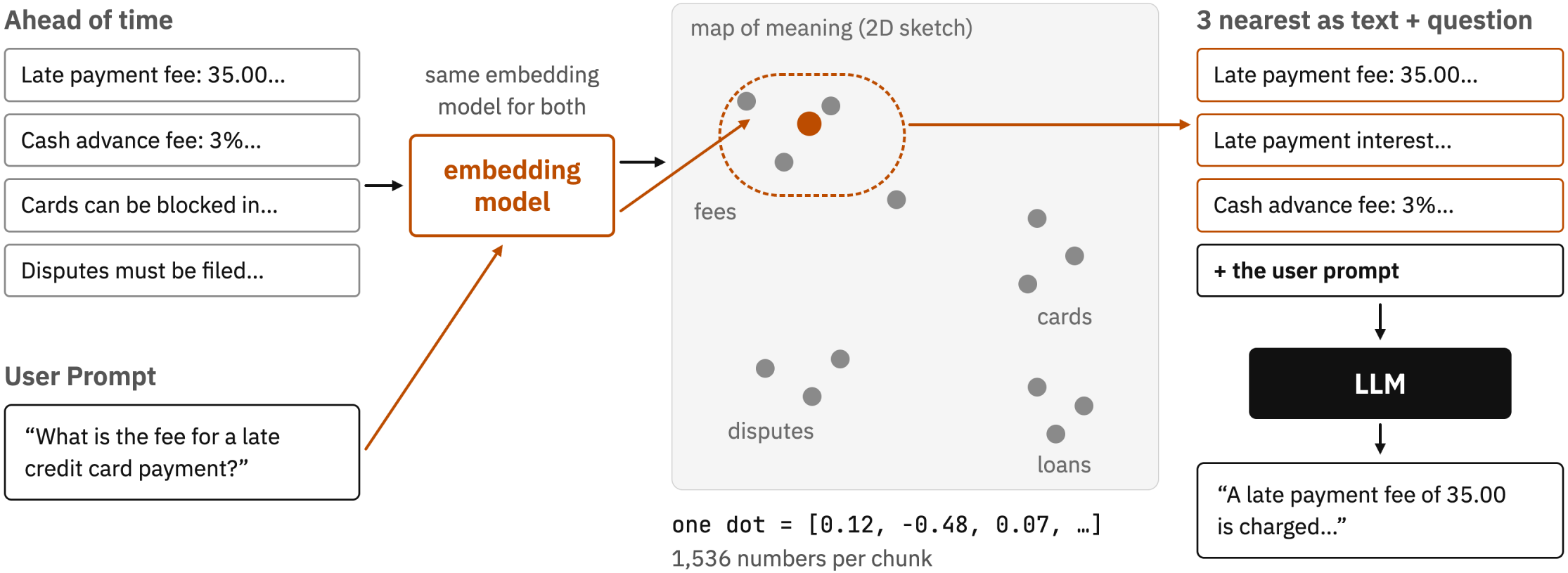


“A late payment fee of 35.00 is charged if the minimum payment isn’t received by the due date.”

Specific, and taken from the bank’s own text.

RAG = search first, then put only what you found into the context.

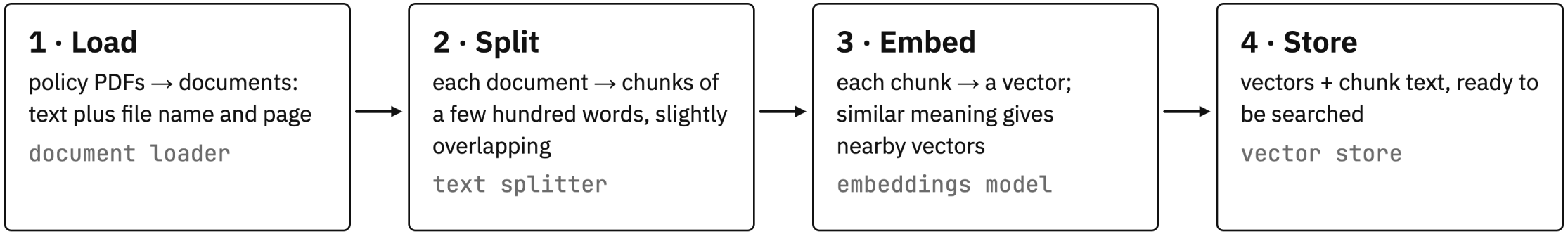
Embeddings turn meaning into positions



The LLM only ever sees text. Vectors are just for finding it.

once, ahead of time

Indexing turns documents into searchable chunks




```
$ pip install langchain-community langchain-text-splitters langchain-openai pypdf
```

```python
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain.embeddings import init_embeddings
from langchain_core.vectorstores import InMemoryVectorStore

docs = PyPDFLoader("card_fees_policy.pdf").load()                          # 1 · load
splitter = RecursiveCharacterTextSplitter(chunk_size=800, chunk_overlap=100)
chunks = splitter.split_documents(docs)                                    # 2 · split
embeddings = init_embeddings("openai:text-embedding-3-small")              # 3 · embed
store = InMemoryVectorStore.from_documents(chunks, embeddings)             # 4 · store
```

A vector database is a search index for meaning

One stored chunk

`id`	`c-0412`
`vector`	`[0.12, -0.48, 0.07, …]` 1,536 numbers, used for search
`text`	“If the minimum payment is not received by the due date, a late payment fee of 35.00 is charged…”

`metadata`	`file: "card_fees_policy.pdf"`
	`page: 4`
	`language: "en"`

A query

```
nearest 3 to the question
where language = "en"  (optional)
```

What it does that a plain list can’t

1 · Finds the nearest, fast

An approximate-search index (ANN) avoids comparing the question with every vector: a tiny bit less exact, far faster on millions.

2 · Can filter by metadata (optional)

Search only chunks with language = en, so an English-speaking customer gets English policy text.

3 · Keeps the data

On disk or on a server, the stored chunks survive restarts. InMemoryVectorStore is gone when the program exits.

4 · Adds, replaces and deletes in place

A new or changed policy? Embed only its chunks; nothing else is touched.

per question

A question runs through four steps in code

1 · Embed
question → vector, same model as the chunks
`embeddings`

→

2 · Search
the store returns the 3 nearest chunks
`vector store`

→

3 · Build messages
rules + chunk text + the question
`messages list`

→

4 · Answer
the LLM writes the reply from that text
`chat model`

```python
question = "What is the fee for a late credit card payment?"
q_vector = embeddings.embed_query(question)                    # 1 · embed
docs = store.similarity_search_by_vector(q_vector, k=3)        # 2 · search
context = "\n\n".join(d.page_content for d in docs)
messages = [                                                   # 3 · build messages
    {"role": "system", "content": "Answer using only the bank policies provided. "
                                  "If they don't contain the answer, say so."},
    {"role": "user", "content": f"Policies:\n{context}\n\nQuestion: {question}"},
]
answer = model.invoke(messages)                                # 4 · answer
print(answer.content)
```

A retriever does the search in one call

Steps 1 and 2 from the previous slide…

```
q_vector = embeddings.embed_query(question)
docs = store.similarity_search_by_vector(q_vector, k=3)
```

…become one call

```
retriever = store.as_retriever(search_kwargs={"k": 3})
docs = retriever.invoke("What is the fee for a late credit card payment?")
for d in docs:
    print(d.metadata["page"], d.page_content[:60])
```

k = how many chunks come back

Behind invoke: embed the question, search the store

A Runnable: invoke, stream, batch, and it fits in a chain

Output

```
4  If the minimum payment is not received by the due date, a late
4  Late payment interest is charged at the overdue rate in additi
7  Cash advances are charged a fee of 3% of the amount.
```

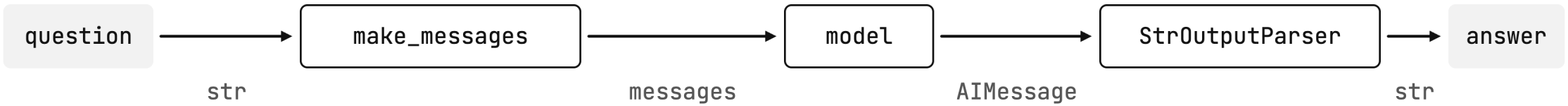
← the distractor from slide 38

Two-step RAG is a short chain

```python
from langchain_core.output_parsers import StrOutputParser

def make_messages(question):
    docs = retriever.invoke(question)                # step 1: retrieve
    context = "\n\n".join(d.page_content for d in docs)
    return [
        {"role": "system", "content": "Answer using only the bank policies provided. "
                                      "If they don't contain the answer, say so."},
        {"role": "user", "content": f"Policies:\n{context}\n\nQuestion: {question}"},
    ]

rag_chain = make_messages | model | StrOutputParser()    # step 2: answer
print(rag_chain.invoke("What is the fee for a late credit card payment?"))
```



Two fixed steps: always retrieve, then answer.

Retrieved chunks become part of the messages

```
[
    {"role": "system", "content": "Answer using only the bank policies provided. "
                                  "If they don't contain the answer, say so."},
    {"role": "user", "content":
        "Policies:\n"
        "[card_fees_policy.pdf, p. 4] A missed minimum payment costs a 35.00 late fee.\n\n"
        "[card_fees_policy.pdf, p. 4] Overdue interest is charged on top of the fee.\n\n"
        "[card_fees_policy.pdf, p. 7] Cash advances cost a fee of 3% of the amount.\n\n"
        "Question: What is the fee for a late credit card payment?"},
]
```

from the retriever

The answer

"A late payment fee of 35.00 is charged if the minimum payment is missed. Overdue interest comes on top."

Cash advance chunk: retrieved but not relevant, a distractor. Extra chunks cost tokens and can mislead, so k stays small.

`labels from metadata: f"[{d.metadata['source']}, p. {d.metadata['page']}] {d.page_content}"`

Five ideas to take away

1	**Talking to a model**	The messages list is the context. invoke returns an AIMessage with the text, token counts and metadata.
2	**Building with LangChain**	| connects steps into a chain. Structured output turns text into checked fields.
3	**Tool calling**	The model asks, your code runs the tool and sends the result back. create_agent runs that loop.
4	**MCP**	One protocol connects any app to any tool server. The LLM never sees it.
5	**RAG**	Search first: embeddings find the relevant chunks, and only their text goes into the messages.

Next: data, agents, workflows, production

Level	Theme	Topics	This module
Level 1	The basics	calls, conversations, streaming, chains, structured output	Covered
Level 2	Data and actions	tools, MCP and retrieval (RAG)	Covered
Level 3	Agents	the agent loop; middleware and memory next	Started
Level 4	LangGraph workflows	custom graphs, multi-agent, human-in-the-loop	
Level 5	Production	tracing, evaluation, deployment, guardrails	